Temperature Logs

Backend architecture for AuditProRx · Express + Node · Prepared 12 October 2026

Who this is for. The developer implementing the API. The Temperature Logs page is already built and runs on browser storage with sample data; this document describes the backend that replaces that storage. It is deliberately small: one table, three routes, four files.

1. What the feature does

A pharmacy builds one temperature sheet per month: one row per day, with morning and evening readings for ambient temperature and one or two fridges, staff initials and a comment per period, plus optional calibration and sign-off details. The sheet is saved to **History**, where it can be viewed, edited again, downloaded as PDF, Excel or CSV, or deleted.

The backend has exactly three jobs:

- Store one log per pharmacy, per location, per month.
- Return a pharmacy's logs for the History tab.
- Delete a log permanently when asked.

Everything else already happens in the browser: building the sheet, filling readings, validation hints, and all three export formats. No export endpoints are needed.

2. Decisions

Decision	Why
One table, JSON columns	A month is at most 31 rows. A separate readings table adds joins for no benefit, and the page already sends this exact shape.
Hard delete, no archive	Deleted means deleted. The pharmacy is responsible for its own records. There is no soft-delete column, no audit table and no restore route.
PUT for save, no POST	Save is an upsert: the first save and every later edit are the same operation on the same month. One idempotent route is safe to retry and needs no separate create call.
List returns full logs	Two years of monthly logs is under 300 KB. View and Edit work from the list, so there is no fetch-by-id route and no computed summary columns.
Exports stay in the browser	PDF, Excel and CSV are generated client-side today. The server never renders a file.
Pharmacy comes from the token	Never from the body or the query string. Reuse the auth middleware and the base-plan check (inventory_reports_access) that other /api routes use.

3. Data model

One table. The three JSON columns hold exactly what the page sends, so nothing is reshaped on the way in or out.

```
create table temperature_logs (
  id            uuid primary key default gen_random_uuid(),
  pharmacy_id   uuid not null references pharmacies(id),
  location_key  text not null,          -- slug of setup.location
```

```
  year          smallint not null,
  month         smallint not null,      -- 1-12 (the page sends 0-11; add 1 on save)
  setup         jsonb not null,         -- month, year, fridges, times, initials, default comments
  rows          jsonb not null,         -- 28-31 day entries, see below
  calibration   jsonb not null,         -- seven sign-off fields
  created_at    timestamptz not null default now(),
  updated_at    timestamptz not null default now(),
  unique (pharmacy_id, location_key, year, month)
);
```

The unique constraint is what makes "one log per month" true on the server. Saving the same month again updates the existing row instead of adding one. On MySQL use JSON columns with the same unique key and `insert ... on duplicate key update`. On MongoDB this is one collection with the same compound unique index and an `updateOne(..., { upsert: true })`.

The JSON shapes

setup — how the sheet was built. `month` is 0–11 here because the page is JavaScript; the `month` column stores it as 1–12.

```
{ "location": "Bergen Road", "year": 2026, "month": 9, "fridges": 1,
  "amTime": "09:30", "pmTime": "17:30",
  "amInitials": "IA", "pmInitials": "IA",
  "amComment": "Within range; alarms normal; no action.",
  "pmComment": "Within range; alarms normal; no action." }
```

rows — one entry per day of the month, in order. Readings are strings so an empty cell stays empty; `fridge2` is always present and is simply empty when the sheet has one fridge.

```
{ "date": "2026-10-01", "weekday": "Thu",
  "am": { "time": "09:30", "ambient": "72.1", "fridge1": "40.3", "fridge2": "",
          "initials": "IA", "comment": "Within range; alarms normal; no action." },
  "pm": { "time": "17:30", "ambient": "73.4", "fridge1": "41.0", "fridge2": "",
          "initials": "IA", "comment": "Within range; alarms normal; no action." } }
```

calibration — all optional, free text.

```
{ "thermometerSn": "", "performedBy": "", "calibrationDate": "", "nextDue": "",
  "certificateNumber": "", "picName": "", "picDate": "" }
```

Validation rules (server side)

Field	Rule
setup.month / setup.year	month 0–11, year a sensible integer
setup.fridges	1 or 2
setup.amTime / pmTime	HH:00 or HH:30; morning before 12:00, evening from 12:00
setup initials	Up to four letters, upper case
rows	Exactly the number of days in that month, dates in sequence from the 1st
rows[].am / pm	time on the 30-minute grid in the right half of the day; readings empty or numeric with at most one decimal; initials up to four letters; comment up to 500 characters
Out-of-range readings	Accepted, never rejected. The sheet exists to record them.

4. API

All routes sit under `/api/temperature-logs`, require the Bearer token, and are gated on the base plan. Responses use named keys like the rest of the API.

Route	Purpose	Response
GET /api/temperature-logs	All logs for this pharmacy, newest month first	{ logs: [{ id, setup, rows, calibration, updatedAt }] }
PUT /api/temperature-logs	Save log: insert, or replace that month if it exists	{ log }
DELETE /api/temperature-logs/:id	Permanent delete	{ ok: true }

Save request

The body is the draft the page already holds. No ids, no timestamps.

```
PUT /api/temperature-logs
Authorization: Bearer <token>

{ "setup": { ... }, "rows": [ ... ], "calibration": { ... } }

200  { "log": { "id": "6d1f...", "setup": { ... }, "rows": [ ... ],
               "calibration": { ... }, "updatedAt": "2026-10-12T10:41:03Z" } }
400  { "errors": [ "rows must have 31 entries for this month" ] }
```

Errors

Status	When
400	Validation failed. The body lists every failing field so the client can show them.
401	Missing or expired token (existing middleware).
403	Plan does not include the feature (existing plan gate).
404	DELETE for an id that does not exist or belongs to another pharmacy. The two cases are not distinguished.

5. Express implementation

Four files plus one mount line. Adapt the `db.query` calls to your existing database helper.

```
src/
  migrations/20261012_temperature_logs.sql   -- the table above
  routes/temperatureLogs.js
  controllers/temperatureLogs.js
  validators/temperatureLog.js
app.js:  app.use("/api/temperature-logs", require("./routes/temperatureLogs"));
```

routes/temperatureLogs.js

```
const router = require("express").Router();
const { list, save, remove } = require("../controllers/temperatureLogs");
const { requireAuth, requirePlan } = require("../middleware/auth"); // existing

router.use(requireAuth, requirePlan("inventory_reports_access"));
router.get("/", list);
router.put("/", save);
router.delete("/:id", remove);
```

```
module.exports = router;
```

controllers/temperatureLogs.js

One gotcha with node-postgres: a JavaScript array passed as a parameter becomes a Postgres array, not JSON. Stringify the three JSON values before passing them.

```
const db = require("../db");
const { validateLog } = require("../validators/temperatureLog");

const slug = (s) =>
  String(s || "").trim().toLowerCase().replace(/[^a-z0-9]+/g, "-")
    .replace(/^-+|-+$/g, "") || "pharmacy";

const COLUMNS = `id, setup, rows, calibration, updated_at as "updatedAt"`;

async function list(req, res, next) {
  try {
    const { rows } = await db.query(
      `select ${COLUMNS} from temperature_logs
        where pharmacy_id = $1
        order by year desc, month desc, location_key`,
      [req.user.pharmacyId],
    );
    res.json({ logs: rows });
  } catch (err) { next(err); }
}

async function save(req, res, next) {
  try {
    const errors = validateLog(req.body);
    if (errors.length) return res.status(400).json({ errors });
    const { setup, rows, calibration } = req.body;
    const { rows: [log] } = await db.query(
      `insert into temperature_logs
         (pharmacy_id, location_key, year, month, setup, rows, calibration)
       values ($1, $2, $3, $4, $5, $6, $7)
       on conflict (pharmacy_id, location_key, year, month)
       do update set setup = excluded.setup, rows = excluded.rows,
                     calibration = excluded.calibration, updated_at = now()
       returning ${COLUMNS}`,
      [req.user.pharmacyId, slug(setup.location), setup.year, setup.month + 1,
       JSON.stringify(setup), JSON.stringify(rows), JSON.stringify(calibration)],
    );
    res.json({ log });
  } catch (err) { next(err); }
}

async function remove(req, res, next) {
  try {
    const { rowCount } = await db.query(
      `delete from temperature_logs where id = $1 and pharmacy_id = $2`,
      [req.params.id, req.user.pharmacyId],
    );
    if (!rowCount) return res.status(404).json({ error: "Log not found" });
    res.json({ ok: true });
  } catch (err) { next(err); }
}

module.exports = { list, save, remove };
```

validators/temperatureLog.js

A plain function, no library. It returns a list of messages; an empty list means the body is valid.

```
const TIME = /^([01]\d|2[0-3]):(00|30)$/;
const TEMP = /^-?\d{1,3}(\.\d)?$/;
```

```
const INITIALS = /^[A-Z]{0,4}$/;
const daysInMonth = (year, month1to12) => new Date(year, month1to12, 0).getDate();

function validateLog(body) {
  const { setup, rows, calibration } = body || {};
  if (!setup || typeof setup !== "object") return ["setup is required"];
  if (!Array.isArray(rows)) return ["rows must be an array"];
  if (!calibration || typeof calibration !== "object") return ["calibration is required"];

  const errors = [];
  const year = Number(setup.year), month = Number(setup.month); // 0-11 from the page
  if (!Number.isInteger(year) || year < 2000 || year > 2100) errors.push("setup.year is invalid");
  if (!Number.isInteger(month) || month < 0 || month > 11) errors.push("setup.month must be 0-11");
  if (![1, 2].includes(setup.fridges)) errors.push("setup.fridges must be 1 or 2");
  if (typeof setup.location !== "string" || setup.location.length > 120)
    errors.push("setup.location is invalid");
  if (!TIME.test(setup.amTime || "") || setup.amTime >= "12:00")
    errors.push("setup.amTime must be HH:00 or HH:30 before 12:00");
  if (!TIME.test(setup.pmTime || "") || setup.pmTime < "12:00")
    errors.push("setup.pmTime must be HH:00 or HH:30 from 12:00");
  for (const k of ["amInitials", "pmInitials"])
    if (!INITIALS.test(setup[k] || "")) errors.push(`setup.${k} must be up to 4 letters`);
  if (errors.length) return errors;

  const expected = daysInMonth(year, month + 1);
  if (rows.length !== expected) return [`rows must have ${expected} entries for this month`];
  const keys = setup.fridges === 2 ? ["ambient", "fridge1", "fridge2"] : ["ambient", "fridge1"];
  const pad = (n) => String(n).padStart(2, "0");

  rows.forEach((row, i) => {
    const date = `${year}-${pad(month + 1)}-${pad(i + 1)}`;
    if (row?.date !== date) errors.push(`rows[${i}].date must be ${date}`);
    for (const p of ["am", "pm"]) {
      const e = row?.[p];
      if (!e) { errors.push(`rows[${i}].${p} is required`); continue; }
      const wrongHalf = p === "am" ? e.time >= "12:00" : e.time < "12:00";
      if (!TIME.test(e.time || "") || wrongHalf) errors.push(`rows[${i}].${p}.time is invalid`);
      for (const k of keys)
        if (e[k] !== "" && !TEMP.test(String(e[k])))
          errors.push(`rows[${i}].${p}.${k} must be a number with at most one decimal`);
      if (!INITIALS.test(e.initials || "")) errors.push(`rows[${i}].${p}.initials is invalid`);
      if (typeof e.comment !== "string" || e.comment.length > 500)
        errors.push(`rows[${i}].${p}.comment is invalid`);
    }
  });
  return errors;
}

module.exports = { validateLog };
```

6. Frontend integration

All browser storage for History goes through four functions in `app/temperature-logs/history.ts`: load, persist, upsert and remove. Swapping them for API calls touches nothing in the UI.

New client file

```
// lib/temperatureLogsApi.ts  (uses the shared axios instance, so token refresh just works)
import api from "@/lib/api";
import type { SavedLog } from "@/app/temperature-logs/history";

type SaveBody = Pick<SavedLog, "setup" | "rows" | "calibration">;

export const listLogs = async (): Promise<SavedLog[]> =>
  (await api.get("/api/temperature-logs")).data.logs;
```

```
export const saveLog = async (body: SaveBody): Promise<SavedLog> =>
  (await api.put("/api/temperature-logs", body)).data.log;

export const deleteLog = async (id: string): Promise<void> => {
  await api.delete(`/api/temperature-logs/${id}`);
};
```

Steps

- History fetches `listLogs()` when its tab opens and shows a skeleton while loading. An error shows a toast with a retry.
- Save log calls `saveLog()`, shows "Saving…" until the response, then the green Saved tick. The returned log replaces the entry in the list, and its server id replaces the client-side month key.
- Delete calls `deleteLog(id)` after the existing confirmation, then removes the card.
- View and Edit keep working from the list, since it already contains the rows.
- The editor draft stays in browser storage, so unsaved work still survives a refresh.
- Remove the seeded sample months and the `persistHistory` call.

7. Tests and rollout

Tests worth writing

- Saving the same month twice leaves one row, with `updated_at` moved forward and `created_at` unchanged.
- Two locations in the same month produce two rows.
- A body with the wrong row count, a bad time, or a non-numeric reading returns 400 with the field named.
- Pharmacy A listing or deleting never touches pharmacy B's logs; the delete returns 404.
- A request without the base plan is refused by the plan gate.

Checklist

Step	Owner
Run the migration	Backend
Add the four files and the mount line	Backend
Run the tests above	Backend
Deploy the API	Backend
Add lib/temperatureLogsApi.ts and swap the four storage functions	Frontend
Remove sample data; verify save, list, edit, delete and all three downloads against the live API	Frontend

Total: one table, three routes, four backend files, one frontend file and a three-function swap.